

LABORATORY OBSERVATION

Course	Full Stack Web Development
Course Code	23CM4121
Experiment No.	5
Student Name	G.Kavya
Roll No.	A24126552082
Date	21-09-2026

1. TITLE

Develop an Interactive Product Calculator Using HTML, CSS and JavaScript

2. AIM

To develop an interactive Product Calculator using HTML, CSS and JavaScript that accepts product name, quantity and unit price, dynamically calculates the total price, and displays the updated result on the web page without refreshing.

3. OBJECTIVE

- To understand the basic structure of an HTML web page.
 - To design a user-friendly interface using CSS.
 - To understand JavaScript functions and variables.
 - To access HTML elements using the Document Object Model (DOM).
 - To handle user input dynamically using JavaScript event listeners.
 - To calculate the total price based on unit price and quantity.
 - To update the displayed result without reloading the page.
-

4. THEORY

HTML (HyperText Markup Language) is used to create the basic structure of the web page. In this application, HTML provides input fields for product name, quantity and unit price, along with the area used to display the calculated total.

CSS (Cascading Style Sheets) is used to design and style the Product Calculator. It controls the layout, colors, spacing, borders, input fields, typography and responsive behavior.

JavaScript provides the dynamic functionality. It reads values entered by the user, performs the calculation through a function, and updates the web page automatically.

The Document Object Model (DOM) represents the HTML document as objects that can be accessed and modified using JavaScript. In this program, `document.getElementById()` is used to access the input and output elements.

The main operations performed by JavaScript are:

- Reading product name, quantity and unit price from input fields.
- Converting quantity and unit price into numbers using `Number()`.
- Calculating the total using: $\text{Total Price} = \text{Quantity} \times \text{Unit Price}$.
- Formatting the result to two decimal places using `toFixed(2)`.
- Updating the total price dynamically through `textContent`.
- Displaying a status message containing the entered product details.
- Using input event listeners to respond immediately to user changes.

Basic flow: User Input → Event Listener → DOM Processing → Calculation → Dynamic Page Update

5. ALGORITHM / PROCEDURE

1. Create a new HTML file for the Product Calculator application.
 2. Define the basic HTML document structure using `html`, `head` and `body` elements.
 3. Add a heading and description for the Product Calculator.
 4. Create an input field to accept the product name.
 5. Create a quantity input field with a default value of 1.
 6. Create a unit-price input field with a default value of 0.00.
 7. Create an output section to display the total price and status message.
 8. Use CSS to style the calculator interface and make it responsive.
 9. Use JavaScript to access the required HTML elements using `document.getElementById()`.
 10. Create a `calculateTotal(quantity, unitPrice)` function to multiply quantity and unit price.
 11. Convert the input values into numbers using `Number()`.
 12. Calculate the total price using $\text{Quantity} \times \text{Unit Price}$.
 13. Display the formatted total using `toFixed(2)`.
 14. Update the product status message dynamically.
 15. Add input event listeners to the product name, quantity and unit price fields.
 16. Call `updateTotal()` whenever an input changes.
 17. Execute the application in a web browser and test different values.
-

6. PROGRAM

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Interactive Product Calculator</title>
  <style>
    :root {
      --ink: #17211b;
      --muted: #65736a;
      --paper: #f8f5ec;
      --panel: #fffd7;
      --line: #d9dfd3;
      --accent: #d95d39;
      --accent-dark: #a53a21;
      --highlight: #e8f0dc;
    }
    * { box-sizing: border-box; }
    body {
      margin: 0;
      min-height: 100vh;
      display: grid;
      place-items: center;
      padding: 24px;
      color: var(--ink);
      background:
        radial-gradient(circle at 15% 15%, #e9efdc 0 14%, transparent 15%),
        linear-gradient(135deg, #f8f5ec 0%, #eaf0e3 100%);
      font-family: Georgia, "Times New Roman", serif;
    }
    main {
      width: min(100%, 760px);
      padding: clamp(28px, 6vw, 62px);
      background: var(--panel);
      border: 1px solid var(--line);
      box-shadow: 14px 14px 0 rgba(23, 33, 27, 0.08);
    }
    .eyebrow {
      margin: 0 0 12px;
      color: var(--accent-dark);
      font: 700 0.78rem/1.2 Arial, sans-serif;
      letter-spacing: 0.14em;
      text-transform: uppercase;
    }
    h1 {
      max-width: 520px;
      margin: 0;
      font-size: clamp(2.4rem, 7vw, 5rem);
      line-height: 0.95;
      font-weight: 500;
    }
    .intro {
      max-width: 500px;
      margin: 20px 0 36px;
      color: var(--muted);
      font-size: 1.08rem;
      line-height: 1.6;
    }
    .calculator { display: grid; gap: 22px; }
    .field-row {
      display: grid;
      grid-template-columns: 1.4fr 1fr 1fr;
      gap: 16px;
    }
    label {
      display: grid;
      gap: 8px;
      color: var(--muted);
      font: 700 0.78rem/1.2 Arial, sans-serif;
```

```

    letter-spacing: 0.08em;
    text-transform: uppercase;
  }
  input {
    width: 100%;
    border: 1px solid var(--line);
    border-radius: 0;
    padding: 14px 12px;
    color: var(--ink);
    background: var(--paper);
    font: 1.05rem Georgia, "Times New Roman", serif;
  }
  input:focus {
    outline: 3px solid rgba(217, 93, 57, 0.22);
    border-color: var(--accent);
  }
  .total {
    display: flex;
    align-items: end;
    justify-content: space-between;
    gap: 20px;
    padding: 22px 0 4px;
    border-top: 2px solid var(--ink);
  }
  .total-label {
    color: var(--muted);
    font: 700 0.8rem/1.2 Arial, sans-serif;
    letter-spacing: 0.1em;
    text-transform: uppercase;
  }
  #total-price {
    color: var(--accent-dark);
    font-size: clamp(2rem, 8vw, 4rem);
    line-height: 1;
  }
  .status {
    margin: 0;
    min-height: 1.3em;
    color: var(--muted);
    font: 0.85rem/1.4 Arial, sans-serif;
  }
  @media (max-width: 600px) {
    .field-row { grid-template-columns: 1fr; }
    .total { align-items: start; flex-direction: column; gap: 10px; }
  }
</style>
</head>
<body>
  <main>
    <p class="eyebrow">CO-1 / Interactive DOM exercise</p>
    <h1>Product price calculator</h1>
    <p class="intro">Enter a product, choose the quantity, and set its unit price. The total is calculated as
you type.</p>
    <section class="calculator" aria-label="Product calculator">
      <div class="field-row">
        <label for="product-name">
          Product name
          <input id="product-name" type="text" placeholder="Example: Notebook">
        </label>
        <label for="quantity">
          Quantity
          <input id="quantity" type="number" min="0" step="1" value="1">
        </label>
        <label for="unit-price">
          Unit price
          <input id="unit-price" type="number" min="0" step="0.01" value="0.00">
        </label>
      </div>
      <div class="total" aria-live="polite">
        <span class="total-label">Total price</span>
        <strong id="total-price">$0.00</strong>
      </div>
    </section>
  </main>
</body>
</html>

```

```

    </div>
    <p class="status" id="status">Enter a product and price to see the total.</p>
  </section>
</main>
<script>
  const productNameInput = document.getElementById("product-name");
  const quantityInput = document.getElementById("quantity");
  const unitPriceInput = document.getElementById("unit-price");
  const totalPrice = document.getElementById("total-price");
  const status = document.getElementById("status");

  function calculateTotal(quantity, unitPrice) {
    return quantity * unitPrice;
  }

  function updateTotal() {
    const productName = productNameInput.value.trim();
    const quantity = Number(quantityInput.value) || 0;
    const unitPrice = Number(unitPriceInput.value) || 0;
    const total = calculateTotal(quantity, unitPrice);

    totalPrice.textContent = `$$${total.toFixed(2)}$`;
    status.textContent = productName
      ? `${quantity} ${productName}${quantity === 1 ? "" : "s"} at $$${unitPrice.toFixed(2)} each`
      : "Enter a product and price to see the total.";
  }

  productNameInput.addEventListener("input", updateTotal);
  quantityInput.addEventListener("input", updateTotal);
  unitPriceInput.addEventListener("input", updateTotal);
</script>
</body>
</html>

```

7. CODE EXPLANATION

Code / Function	Explanation
<!DOCTYPE html>	Defines the document as an HTML5 document.
<meta name="viewport">	Makes the web page responsive on different screen sizes.
.field-row	Arranges the input fields in a responsive grid.
input	Accepts product name, quantity and unit price from the user.
document.getElementById()	Accesses HTML elements through the DOM.
Number()	Converts input values into numerical values.
calculateTotal()	Calculates the total using quantity $\times$ unit price.
toFixed(2)	Displays the calculated amount with two decimal places.
textContent	Dynamically changes the text displayed on the page.
addEventListener()	Executes a function when a specified event occurs.
"input"	Detects changes made in the input fields.
trim()	Removes leading and trailing spaces from the product name.
updateTotal()	Reads inputs, calculates the total and updates the page.

8. TEST CASES AND EXECUTION DATA

Test Case	Product Name	Unit Price	Quantity	Expected Result
TC-01	Notebook	100	1	Total should be \$100.00
TC-02	Pen	50	2	Total should be \$100.00
TC-03	Bag	750	3	Total should be \$2250.00
TC-04	Book	250	5	Total should be \$1250.00

9. OUTPUT

The webpage displays the Product Price Calculator interface. When the user enters a product name, quantity and unit price, the total price is updated immediately without refreshing the page.

TC-01:

CO-1 / INTERACTIVE DOM EXERCISE

Product price calculator

Enter a product, choose the quantity, and set its unit price. The total is calculated as you type.

PRODUCT NAME	QUANTITY	UNIT PRICE
Notebook	1	100

\$100.00

TOTAL PRICE

1 Notebook at \$100.00 each

TC-02:

CO-1 / INTERACTIVE DOM EXERCISE

Product price calculator

Enter a product, choose the quantity, and set its unit price. The total is calculated as you type.

PRODUCT NAME	QUANTITY	UNIT PRICE
Pen	2	50

TOTAL PRICE

\$100.00

2 Pens at \$50.00 each

TC-03:

CO-1 / INTERACTIVE DOM EXERCISE

Product price calculator

Enter a product, choose the quantity, and set its unit price. The total is calculated as you type.

PRODUCT NAME	QUANTITY	UNIT PRICE
Bag	3	750

TOTAL PRICE

\$2250.00

3 Bags at \$750.00 each

TC-04:

CO-1 / INTERACTIVE DOM EXERCISE

Product price calculator

Enter a product, choose the quantity, and set its unit price. The total is calculated as you type.

PRODUCT NAME	QUANTITY	UNIT PRICE
Book	5	250

TOTAL PRICE

5 Books at \$250.00 each

\$1250.00

10. RESULT

Thus, an interactive Product Calculator was successfully developed using HTML, CSS and JavaScript. The application accepts product name, quantity and unit price, calculates the total price dynamically, and updates the webpage using DOM manipulation, JavaScript functions and event listeners.